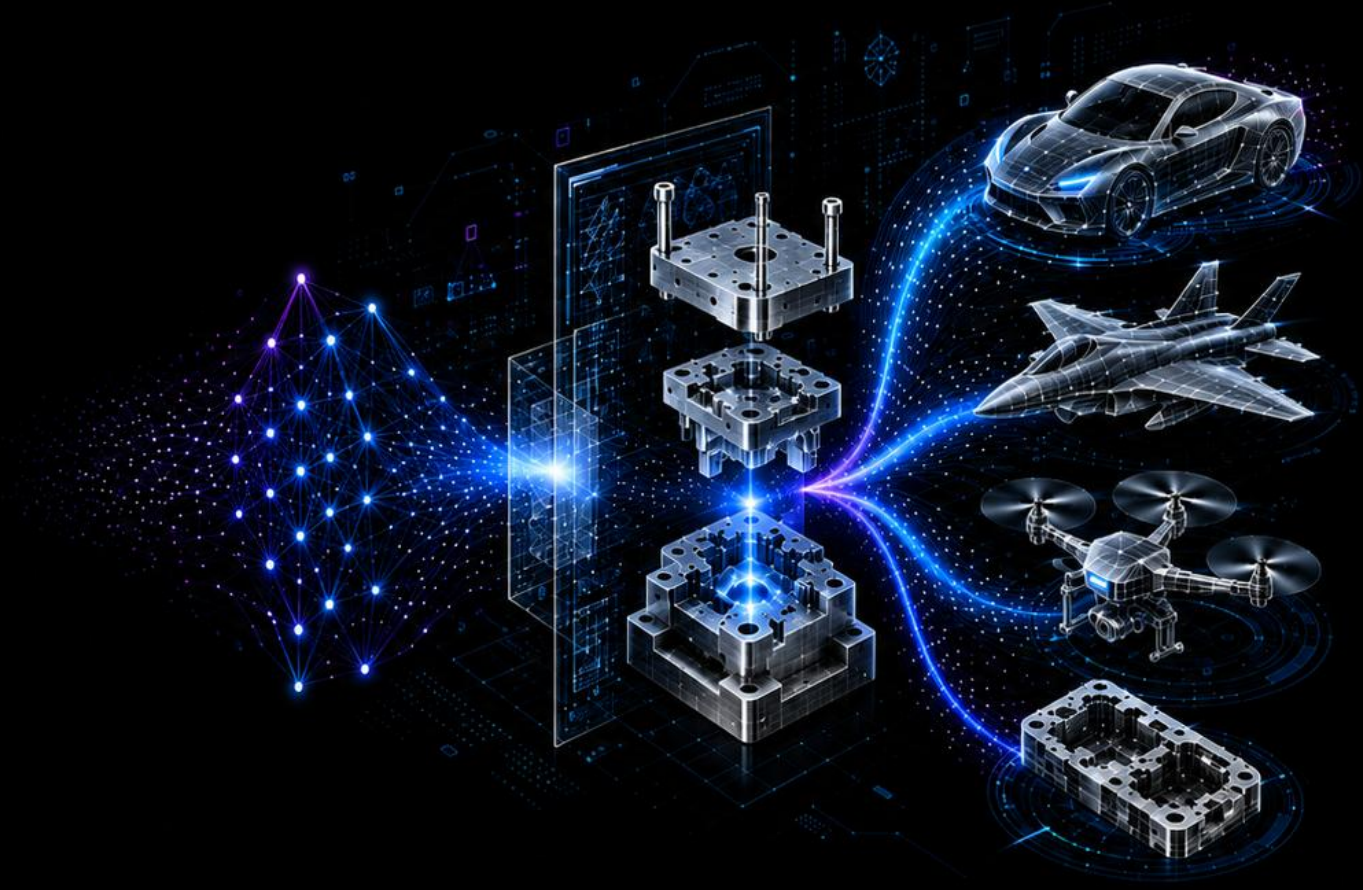
4장 스터디

스터디 보고



정승한
(지도교수: 양선웅)
한양대학교 ERICA 기계공학과

Intelligent Design for Engineering Applications Lab

4.1 인공 신경망의 한계와 딥러닝 출현

Intelligent Design for Engineering Applications Lab

Part 1 CH.4.1

Part 2 CH.4.2

Part 3 CH.4.3

MLP의 등장

- 3가지 게이트에 대해 알아보고 MLP 등장 이유를 확인
- MLP : Multi-layer perceptron, 비선형 분류 데이터도 학습 가능하며 DNN (Deep Neural Network)라고도 함

기본 개념

❖ Perceptron [figure.1]

✓ 인공 신경망에서 이용하는 구조

✓ $w_1x_1 + w_2x_2 = y$ (Basic perceptron)

❖ AND 게이트 [figure.2]

✓ 모든 입력이 1일 때 작동

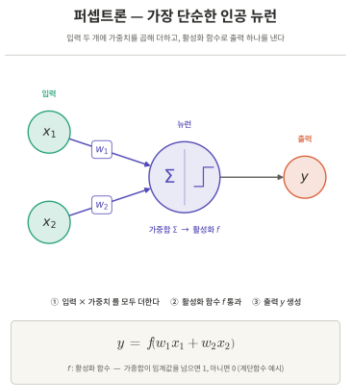
❖ OR 게이트 [figure.3]

✓ 입력 모두가 0을 갖는 경우를 제외한 나머지에 작동

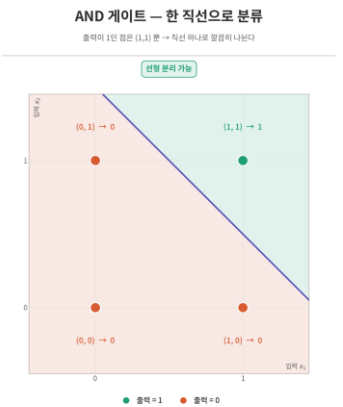
❖ XOR 게이트 [figure.4]

✓ 하나만 1을 갖는 경우일 때 작동

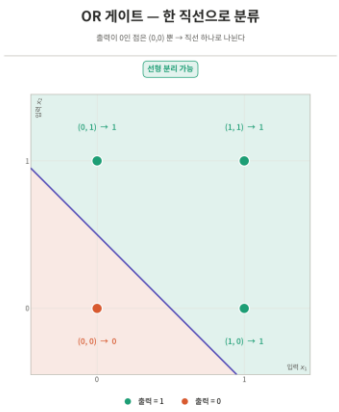
✓ 비선형 분류이므로 MLP 필요 (일반 perceptron으로는 분류 불가)



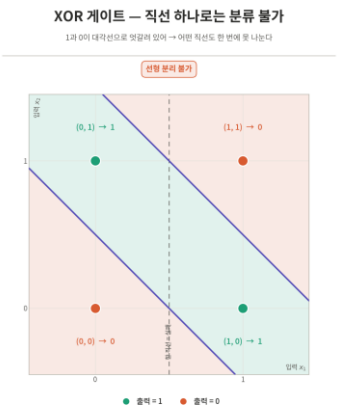
[figure.1]



[figure.2]



[figure.3]



[figure.4]

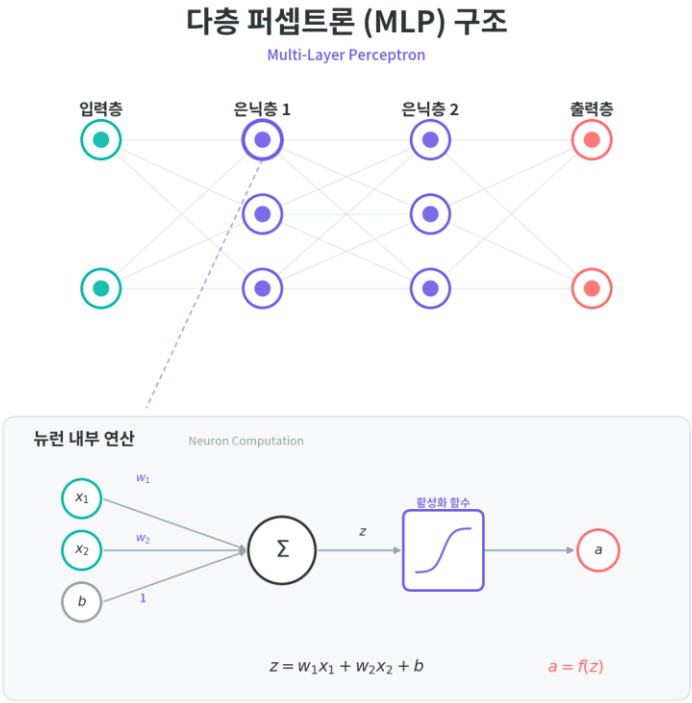
[1] ref1
[2] ref2

4.2 딥러닝 구조

Deep Learning 용어

- 신경망을 이루는 구성 요소에 대해 하나씩 살펴보기
- 딥러닝은 입력층, 출력층과 두 개 이상의 은닉층으로 구성

구분	구성 요소	설명
Layer	Input layer	데이터를 받아들이는 층
	Hidden layer	모든 입력으로부터 값을 받아 가중합을 계산하고 활성화 함수에 적용하여 출력층으로 전달하는 층
	Output layer	신경망의 최종 결과값이 포함된 층
Weight		노드와 노드 간 연결 강도
Bias		가중합에 더해주는 상수, 출력 값을 조절함
Weighted sum		가중치와 신호의 곱
함수	Activation function	신호를 받아 적절히 처리하여 출력하는 함수
	Loss function	가중치 학습을 위해 출력과 실제 값 간의 오차 측정 함수



[1] ref1
[2] ref2

4.2 딥러닝 구조

구성 요소

- 가중치, 가중합, 활성화 함수에 대한 설명

기본 개념

❖ Weight (가중치)

✓ Weight는 입력 값이 연산 결과에 미치는 영향을 조절하는 요소

❖ Weighted sum (가중치)

✓ 각 노드로 들어오는 신호에 가중치를 곱해서 다음 노드로 전달됨

❖ Activation function (활성화 함수)

✓ 전달 함수에서 전달 받은 값을 출력할 때 출력 값을 변화시키는 비선형 함수

1. Sigmoid
2. Hyperbolic tangent
3. ReLU
4. Leaky ReLU

Activation function

❖ Sigmoid

✓ 결과를 0~1 사이에서 비선형 형태로 변형

✓ $f(x) = \frac{1}{1+e^{-x}}$

✓ 모델이 깊어지면 기울기가 사라지는 '기울기 소멸 문제 ' 존재

❖ Hyperbolic tangent

✓ 결과를 −1~1 사이에서 비선형 형태로 변형

✓ Sigmoid에서 결과의 평균이 양수로 편향되는 문제는 해결

✓ 하지만 기울기 소멸 문제는 여전

❖ ReLU

✓ 입력이 음수일 경우 0 출력, 양수일 때 입력 값 출력

✓ Gradient descent에 영향을 주지 않아 학습 빠름

✓ 기울기 소멸 문제 발생하지 않음

✓ 죽은 ReLU 문제 발생 (음수는 항상 0으로 출력)

❖ Leaky ReLU

✓ 입력 값이 음수이면 0.001과 같이 매우 작은 수로 변환

✓ 입력 값이 수렴하는 구간 제거, 죽은 ReLU 문제 해결 가능

[1] ref1
[2] ref2

4.2 딥러닝 구조

구성 요소

- 추가적인 activation function 소개

Activation function

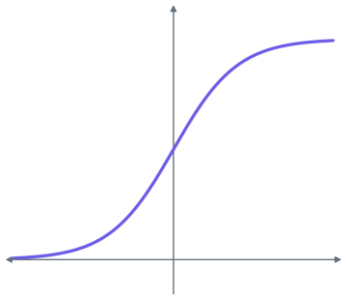
❖ Softmax function

- ✓ 입력 값을 0~1 사이에 출력되도록 정규화
- ✓ 출력 값들의 총합이 항상 1이 되도록
- ✓ 딥러닝 출력 노드의 활성화 함수로 많이 사용
- ✓ $y_k = \frac{\exp(a_k)}{\sum_{i=1}^{n} \exp(a_k)}$
- ✓ exp는 지수 함수이며 n은 출력층의 뉴런 개수
- ✓ y_k는 그중 k번째 출력을 의미

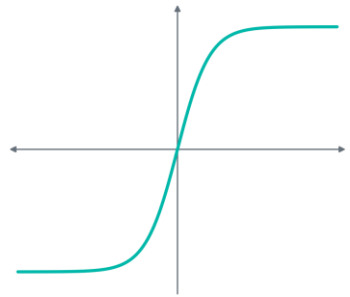
❖ Softmax Function 특징

- ✓ 입력벡터를 지수를 씌워 양수로 만든 후 전체 합으로 나누어 정규화하는 구조
- ✓ 너무 큰 입력 값을 주면 overflow 발생 가능 (z가 1000일 때)
- ✓ Softmax Function을 쓰더라도 큰 값이 그대로 보존
- ✓ 모든 입력에 같은 상수 c를 더하더라도 결과 불변
- ✓ Hardmax function의 경우 0,1을 배정하지만 softmax의 경우 경계를 매끄럽게 만들어 경사 하강법으로 학습할 수 있음

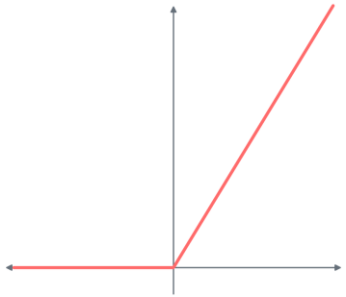
Sigmoid



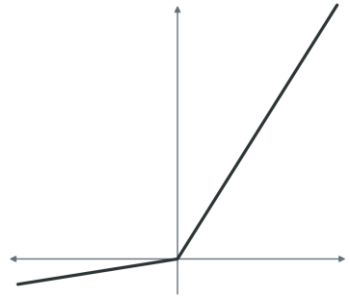
Hyperbolic Tangent



ReLU



Leaky ReLU



[1] ref1
[2] ref2

Loss Function

- 경사 하강법은 learning rate와 loss function의 순간 기울기를 이용하여 가중치를 업데이트
- 미분의 기울기를 이용해 오차를 비교, 최소화하는 방향으로 이동시키는 방법 -> 이때 오차를 구하는 방법이 loss function

Loss function

❖ MSE : Mean Squared Error

- ✓ 실제 값과 예측 값의 차이를 제곱하여 평균을 냄
- ✓ 실제 값과 예측 값의 차이가 작아질수록 예측력이 좋다는 의미
- ✓ $MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$

❖ CEE : Cross Entropy Error

- ✓ Classification 문제에서 one-hot encoding 했을 때 사용할 수 있는 계산법
- ✓ 일반적으로 MSE와 Sigmoid function을 결합하면 기울기가 매끄럽지 못하며 학습 속도 느림
- ✓ CEE는 두 개의 확률 분포 차이를 이용하기 때문에 Sigmoid의 영향을 덜 받음
- ✓ $CEE = -\sum_{i=1}^{n} y_i log\hat{y}_i$

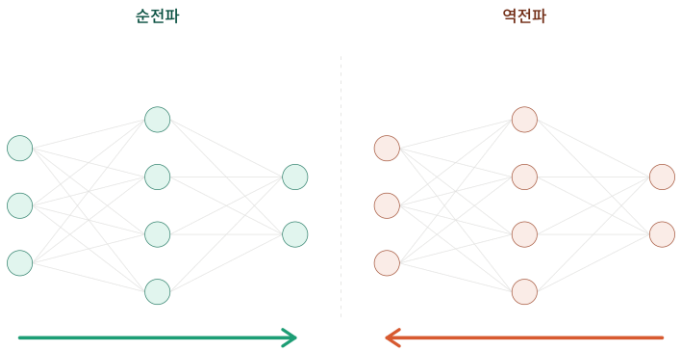
Forward/Backward-propagation

❖ Feed Forward

- ✓ 네트워크에 훈련 데이터가 들어올 때 발생
- ✓ 다음 층의 뉴런으로 전송하는 방식

❖ Backward-propagation

- ✓ Loss가 계산되면 역으로 전파
- ✓ 예측 값과 실제 값 차이를 각 뉴런의 가중치로 미분한 후 기존 가중치 값에서 뺌



[1] ref1
[2] ref2

4.2 딥러닝 구조

딥러닝의 문제점과 해결방안

- 딥러닝이란 활성화 함수가 적용된 여러 은닉층을 결합하여 비선형 영역을 표현하는 것
- 하지만 은닉층이 많을수록 문제점이 발생

문제점

❖ Over-fitting 문제

- ✓ Over-fitting은 훈련 데이터를 과하게 학습해서 발생
- ✓ 훈련 데이터 값과 예측 값의 오차는 줄어들지만 검증 데이터에서는 오차 증가

❖ 기울기 소멸 문제

- ✓ 은닉층이 많은 신경망에서 출력 -〉 은닉층으로 전달되는 오차가 크게 줄어들어 학습 x
- ✓ Sigmoid나 Hyperbolic tangent 대신 ReLU 사용하면 해결할 수 있음

❖ 성능이 나빠지는 문제

- ✓ 경사 하강법은 Loss function의 비용이 최소가 되는 지점을 찾을 때까지 계속 이동시킴
- ✓ 이때 성능이 나빠지는 문제가 발생 – 기울기가 0에 수렴하면서 학습속도 저하

성능 개선 방안

❖ Stochastic Gradient Descent

- ✓ 임의로 선택한 데이터에 대해 기울기 계산
- ✓ 속도 빠르지만 불안정

❖ Batch Gradient Descent

- ✓ 전체 데이터의 loss 기울기를 평균 내어 1회 수정
- ✓ 한 스텝에 모든 데이터셋을 사용하여 느림
- ✓ $w = w - \eta \frac{1}{m} \sum_{i=1}^{m} \frac{dL}{dw}$

❖ Mini-Batch Gradient Descent

- ✓ 한 개의 데이터를 mini-batch 여러 개로 나누고 mini-batch 들의 평균 기울기를 이용하여 모델 업데이트

[1] ref1
[2] ref2

4.3 딥러닝 알고리즘

신경망

- 딥러닝 알고리즘은 심층 신경망을 사용한다는 공통점 존재
- 목적에 따라 CNN, RNN, RBM, DBN 존재

DNN : Deep Neural Network

❖ 다수의 은닉층이 존재

- ✓ 별도의 트릭 없이 비선형 분류 가능
- ✓ 연산 량이 많고 기울기 소멸 문제 발생 가능
- ✓ 적절한 activation function 요구됨

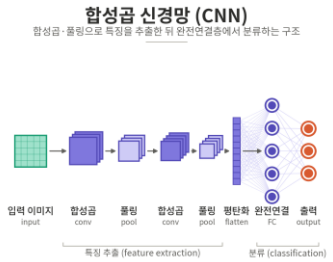


CNN : Convolutional Neural Network

❖ 이미지 처리 성능 우수

❖ DNN과 비교되는 차별성

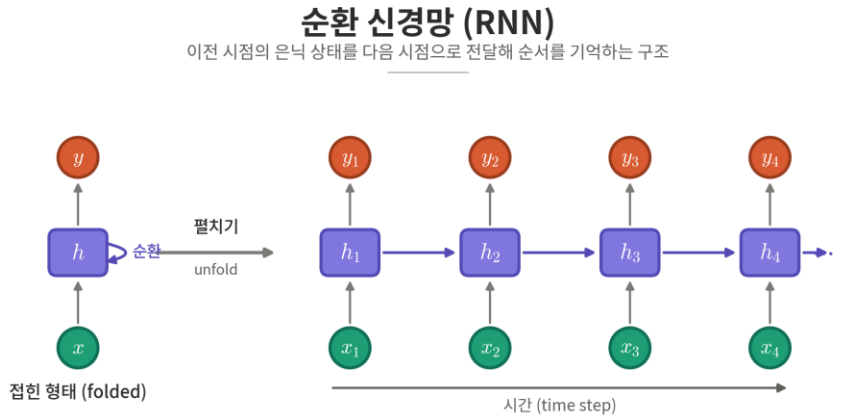
- ✓ 각 층의 입출력 형상유지
- ✓ 이미지의 공간 정보 유지
- ✓ 인접 이미지와 차이점 효과적으로 인식
- ✓ 필터가 존재함으로 인해 학습 파라미터가 적음



RNN : Recurrent Neural Network

❖ 시간 흐름에 따라 변화하는 데이터를 학습하기 위한 신경망

- ✓ '순환'은 자신을 참조한다는 것
- ✓ 현재 결과가 이전 결과와 연관이 있음을 인식
- ✓ Temporal property를 가진 데이터가 많음
- ✓ 데이터는 동적이고 길이가 가변적인 경우가 다수



[1] ref1
[2] ref2

4.3 딥러닝 알고리즘

신경망

- 딥러닝 알고리즘은 심층 신경망을 사용한다는 공통점 존재
- 목적에 따라 CNN, RNN, RBM, DBN 존재

Boltzmann machine

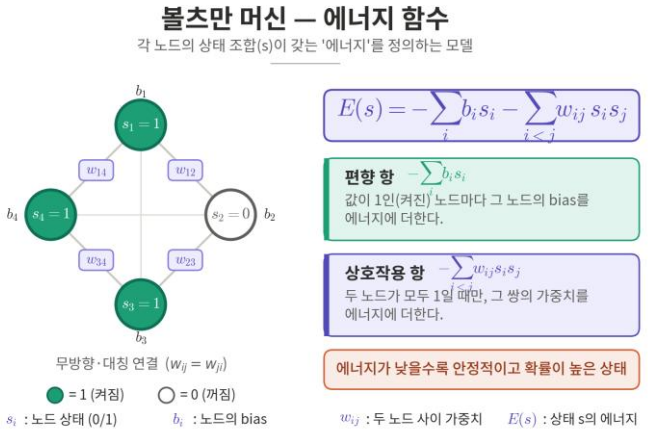
❖ 볼츠만 머신은 확률적 생성 모델이자 에너지 기반 모델

✓ $E(s) = -\sum b_i s_i - \sum w_{ij} s_i s_j$ (에너지 함수)

✓ $P(s) = \frac{e^{-E(s)}}{Z}$

✓ 에너지가 낮은 상태일수록 확률이 높음

✓ 데이터가 실제로 나타나는 확률 높이도록 (MLE와 동일한 목표)



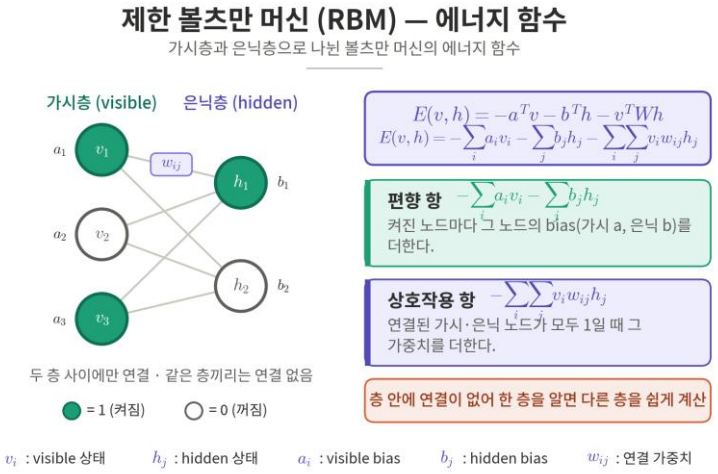
Restricted Boltzmann machine

❖ 제한된 볼츠만 머신은 기존 볼츠만 머신에서 같은 층 내부 연결 제한

✓ RBM에서는 visible layer, hidden layer 내부의 연결은 제한됨

✓ $E(v,h) = -a^T v - b^T h - v^T W h$

✓ 같은 층 내부 연결이 없으므로, visible이 주어질 때 hidden unit들이 독립



[1] ref1
[2] ref2

Any Questions?

이름 정 승 한

소속 한양대학교 기계공학과

이메일 jshan0407@hanyang.ac.kr

Intelligent Design for Engineering Applications Lab

